

Sistemas Operativos

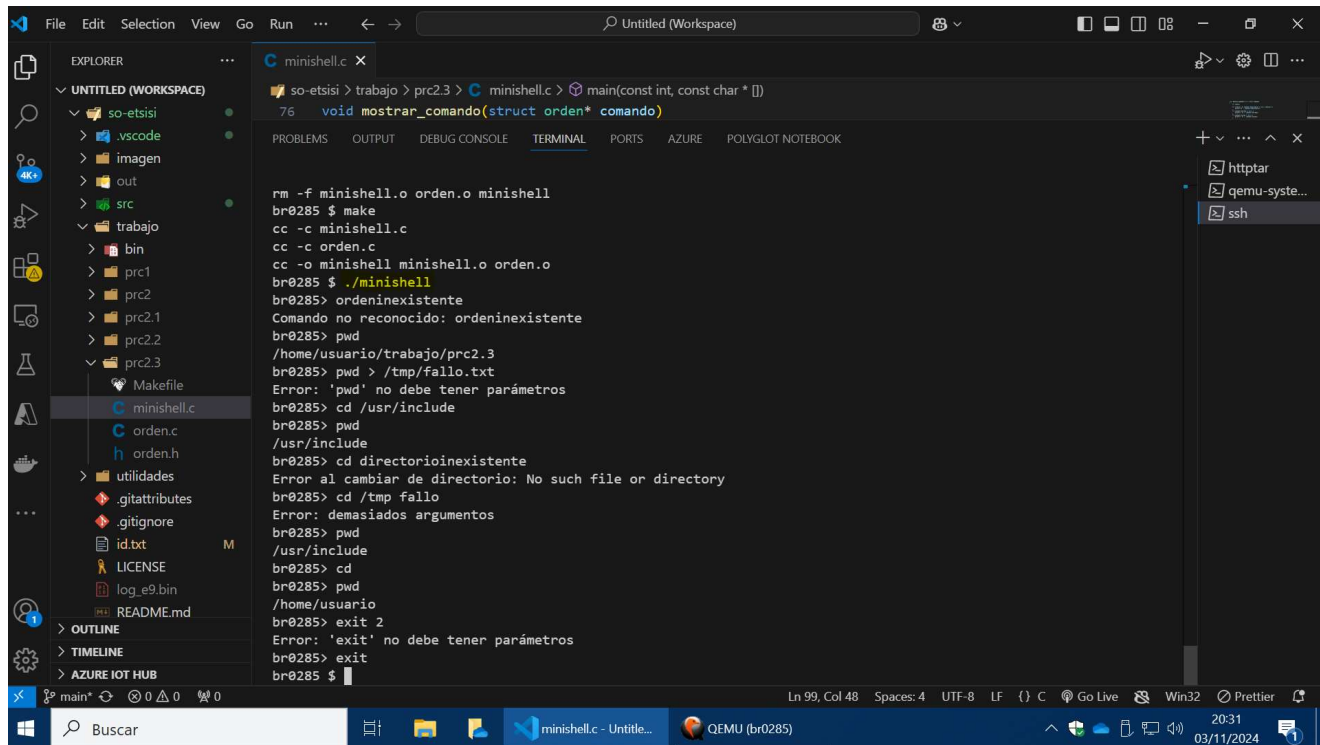
Práctica 2.3

Fecha: 5/11/2024

Alumno: Yzan Martinez Manzano

Correo: yzan.martinez.manzano@alumnos.upm.es

1. Órdenes internas



```
so-etsisi > trabajo > prc2.3 > C minishell.c > main(const int, const char * [])
76 void mostrar_comando(struct orden* comando)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE POLYGLOT NOTEBOOK

rm -f minishell.o orden.o minishell
br0285 $ make
cc -c minishell.c
cc -c orden.c
cc -o minishell minishell.o orden.o
br0285 $ ./minishell
br0285 > ordeninexistente
Comando no reconocido: ordeninexistente
br0285 > pwd
/home/usuario/trabajo/prc2.3
br0285 > pwd > /tmp/fallo.txt
Error: 'pwd' no debe tener parámetros
br0285 > cd /usr/include
br0285 > pwd
/usr/include
br0285 > cd directorioinexistente
Error al cambiar de directorio: No such file or directory
br0285 > cd /tmp fallo
Error: demasiados argumentos
br0285 > pwd
/usr/include
br0285 > cd
br0285 > pwd
/home/usuario
br0285 > exit 2
Error: 'exit' no debe tener parámetros
br0285 > exit
br0285 $
```

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include "orden.h"

int ejecutar_pwd(struct orden* comando)
{
    char directorio[250];
    if (comando->args[1] != NULL || comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'pwd' no debe tener parámetros\n");
        return -1;
    }
    if (getcwd(directorio, sizeof(directorio)) != NULL) {
        printf("%s\n", directorio);
        return 0;
    } else {
```

```

        perror("No se pudo obtener el directorio actual");
        return -1;
    }
}

int ejecutar_cd(struct orden* comando)
{
    const char* ruta_home = getenv("HOME");

    if (comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'cd' no permite redirección.\n");
        return -1;
    }

    if (comando->args[1] == NULL) {
        if (ruta_home == NULL) {
            fprintf(stderr, "Error en la variable HOME\n");
            return -1;
        }
        if (chdir(ruta_home) != 0) {
            perror("Error al cambiar de directorio");
            return -1;
        }
    } else if (comando->args[2] != NULL) {
        fprintf(stderr, "Error: demasiados argumentos\n");
        return -1;
    } else {
        if (chdir(comando->args[1]) != 0) {
            perror("Error al cambiar de directorio");
            return -1;
        }
        return 0;
    }
}

int ejecutar_exit(struct orden* comando)
{
    if (comando->args[1] != NULL || comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'exit' no debe tener parámetros\n");
        return -1;
    }
    return 1;
}

int procesar_comando(struct orden* comando)
{
    if (strcmp("pwd", comando->args[0]) == 0) {
        return ejecutar_pwd(comando);
    } else if (strcmp("cd", comando->args[0]) == 0) {
        return ejecutar_cd(comando);
    } else if (strcmp("exit", comando->args[0]) == 0) {
        return ejecutar_exit(comando);
    } else {
        fprintf(stderr, "Comando no reconocido: %s\n", comando->args[0]);
    }
}

```

```

        return -1;
    }
}

void mostrar_comando(struct orden* comando)
{
    int indice;

    for (indice = 0; comando->args[indice] != NULL; indice++) {
        printf("%s ", comando->args[indice]);
    }
    if (comando->entrada != NULL) {
        printf("< %s ", comando->entrada);
    }
    if (comando->salida != NULL) {
        printf("> %s ", comando->salida);
    }
    printf("\n");
}

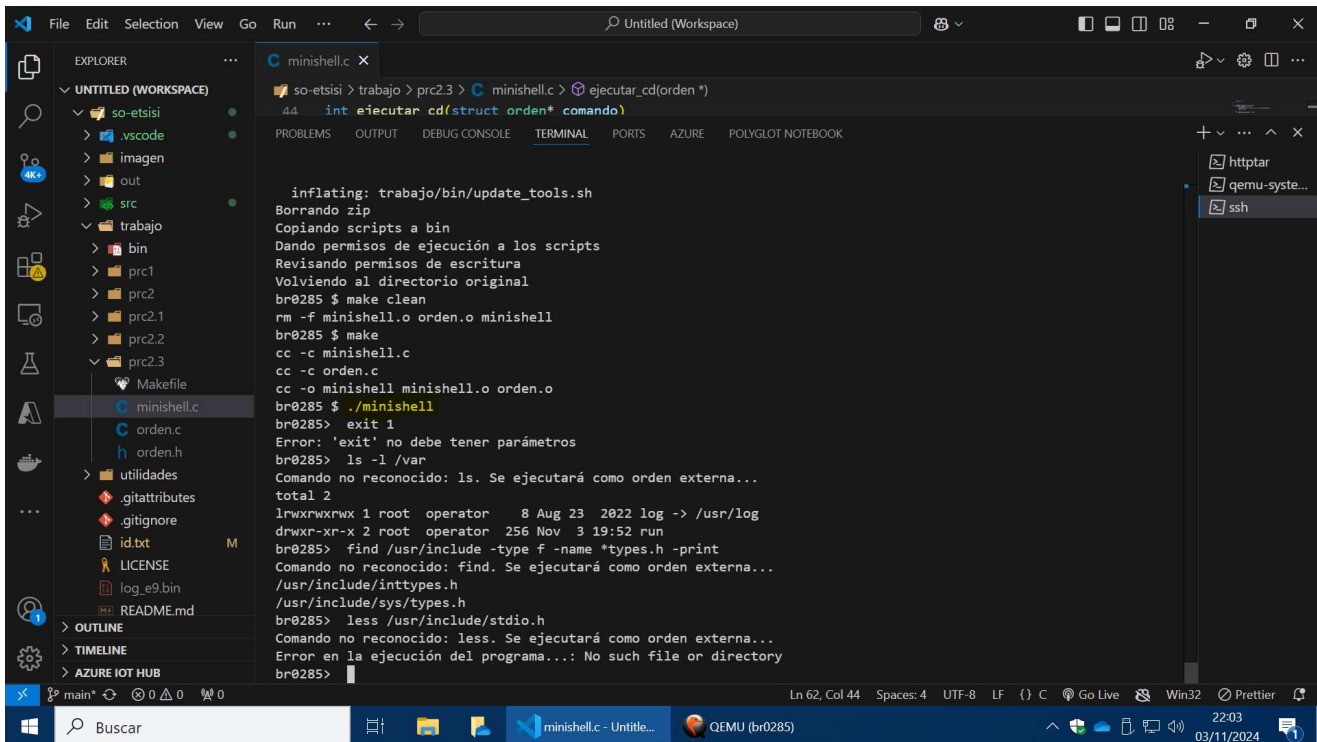
int main(const int num_args, const char* args[])
{
    struct orden* comando_actual;
    int estado;
    int continuar = 1;

    do {
        comando_actual = leer_orden("br0285>");
        if (comando_actual->args[0] != NULL) {
            estado = procesar_comando(comando_actual);
            if (estado == 1 && strcmp("exit", comando_actual->args[0]) == 0) {
                continuar = 0;
            }
        }
    } while (continuar);

    return 0;
}

```

2. Órdenes externas



```
so-etsisi > trabajo > prc2.3 > C minishell.c > ejecutar_cd(orden *)
44 int ejecutar_cd(struct orden* comando)

    inflating: trabajo/bin/update_tools.sh
Borrando zip
Copiando scripts a bin
Dando permisos de ejecución a los scripts
Revisando permisos de escritura
Volviendo al directorio original
br0285 $ make clean
rm -f minishell.o orden.o minishell
br0285 $ make
cc -c minishell.c
cc -c orden.c
cc -o minishell minishell.o orden.o
br0285 $ ./minishell
br0285> exit 1
Error: 'exit' no debe tener parámetros
br0285> ls -l /var
Comando no reconocido: ls. Se ejecutará como orden externa...
total 2
lrwxrwxrwx 1 root operator 8 Aug 23 2022 log -> /usr/log
drwxr-xr-x 2 root operator 256 Nov 3 19:52 run
br0285> find /usr/include -type f -name *types.h -print
Comando no reconocido: find. Se ejecutará como orden externa...
/usr/include/inttypes.h
/usr/include/sys/types.h
br0285> less /usr/include/stdio.h
Comando no reconocido: less. Se ejecutará como orden externa...
Error en la ejecución del programa...: No such file or directory
br0285>
```

```
#include <stdio.h>
#include <string.h>
#include "orden.h"
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/types.h>

void ejecutar_orden_externa(struct orden* comando)
{
    pid_t proceso_id = fork();
    int estado;

    if (proceso_id < 0) {
        perror("Error al crear el proceso...");
        return;
    } else if (proceso_id == 0) {
        execvp(comando->args[0], comando->args);
        perror("Error en la ejecución del programa...");
        exit(EXIT_FAILURE);
    } else {
        waitpid(proceso_id, &estado, 0);
    }
}

int ejecutar_pwd(struct orden* comando)
{
    char directorio[250];

    if (comando->args[1] != NULL || comando->entrada != NULL || comando->salida != NULL) {
```

```

        fprintf(stderr, "Error: 'pwd' no debe tener parámetros\n");
        return -1;
    }

    if (getcwd(directorio, sizeof(directorio)) != NULL) {
        printf("%s\n", directorio);
        return 0;
    } else {
        perror("No se pudo obtener el directorio actual");
        return -1;
    }
}

int ejecutar_cd(struct orden* comando)
{
    const char* ruta_home = getenv("HOME");

    if (comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'cd' no permite redirección.\n");
        return -1;
    }

    if (comando->args[1] == NULL) {
        if (ruta_home == NULL) {
            fprintf(stderr, "Error en la variable HOME\n");
            return -1;
        }
        if (chdir(ruta_home) != 0) {
            perror("Error al cambiar de directorio");
            return -1;
        }
    } else if (comando->args[2] != NULL) {
        fprintf(stderr, "Error: demasiados argumentos\n");
        return -1;
    } else {
        if (chdir(comando->args[1]) != 0) {
            perror("Error al cambiar de directorio");
            return -1;
        }
        return 0;
    }
}

int ejecutar_exit(struct orden* comando)
{
    if (comando->args[1] != NULL || comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'exit' no debe tener parámetros\n");
        return -1;
    }

    return 1;
}

int procesar_comando(struct orden* comando)

```

```

{
    if (strcmp("pwd", comando->args[0]) == 0) {
        return ejecutar_pwd(comando);
    } else if (strcmp("cd", comando->args[0]) == 0) {
        return ejecutar_cd(comando);
    } else if (strcmp("exit", comando->args[0]) == 0) {
        return ejecutar_exit(comando);
    } else {
        fprintf(stderr, "Comando no reconocido: %s. Se ejecutará como orden externa...\n",
comando->args[0]);
        ejecutar_orden_externa(comando);
        return 0;
    }
}

void mostrar_comando(struct orden* comando)
{
    int indice;

    for (indice = 0; comando->args[indice] != NULL; indice++) {
        printf("%s ", comando->args[indice]);
    }
    if (comando->entrada != NULL) {
        printf("< %s ", comando->entrada);
    }
    if (comando->salida != NULL) {
        printf("> %s ", comando->salida);
    }
    printf("\n");
}

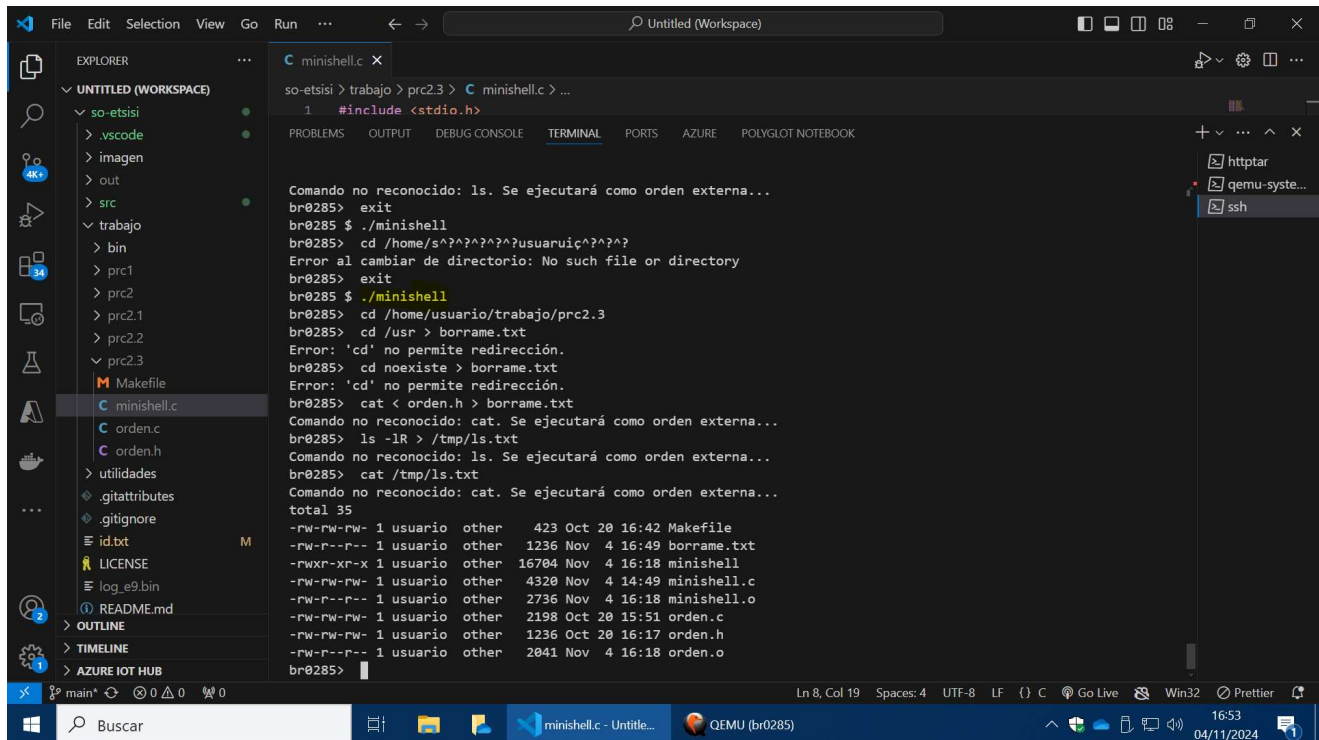
int main(const int num_args, const char* args[])
{
    struct orden* comando_actual;
    int estado, continuar = 1;

    do {
        comando_actual = leer_orden("br0285> ");
        if (comando_actual->args[0] != NULL) {
            estado = procesar_comando(comando_actual);
            if (estado == 1 && strcmp("exit", comando_actual->args[0]) == 0) {
                continuar = 0;
            }
        }
    } while (continuar);

    return 0;
}

```

3. Redirección de E/S



```
so-etsisi > trabajo > prc2.3 > C minishell.c ...
1 #include <stdio.h>

Comando no reconocido: ls. Se ejecutará como orden externa...
br0285> exit
br0285 $ ./minishell
br0285> cd /home/s^?^?^?^?^?usuarui^?^?^?
Error al cambiar de directorio: No such file or directory
br0285> exit
br0285 $ ./minishell
br0285> cd /home/usuario/trabajo/prc2.3
br0285> cd /usr > borrame.txt
Error: 'cd' no permite redirección.
br0285> cd noexiste > borrame.txt
Error: 'cd' no permite redirección.
br0285> cat < orden.h > borrame.txt
Comando no reconocido: cat. Se ejecutará como orden externa...
br0285> ls -lR > /tmp/ls.txt
Comando no reconocido: ls. Se ejecutará como orden externa...
br0285> cat /tmp/ls.txt
Comando no reconocido: cat. Se ejecutará como orden externa...
total 35
-rw-rw-rw- 1 usuario other 423 Oct 20 16:42 Makefile
-rw-r--r-- 1 usuario other 1236 Nov 4 16:49 borrame.txt
-rwxr-xr-x 1 usuario other 16704 Nov 4 16:18 minishell
-rw-rw-rw- 1 usuario other 4320 Nov 4 14:49 minishell.c
-rw-r--r-- 1 usuario other 2736 Nov 4 16:18 minishell.o
-rw-rw-rw- 1 usuario other 2198 Oct 20 15:51 orden.c
-rw-rw-rw- 1 usuario other 1236 Oct 20 16:17 orden.h
-rw-r--r-- 1 usuario other 2041 Nov 4 16:18 orden.o
br0285>
```

```
#include <stdio.h>
#include <string.h>
#include "orden.h"
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <fcntl.h>

void ejecutar_orden_externa(struct orden* comando)
{
    pid_t proceso_id = fork();
    int estado;

    if (proceso_id < 0) {
        perror("Error al crear el proceso...");
        return;
    } else if (proceso_id == 0) {

        if (comando->entrada != NULL) {
            int archivo_entrada = open(comando->entrada, O_RDONLY);
            if (archivo_entrada < 0) {
                perror("Error al abrir el archivo de entrada...");
                exit(EXIT_FAILURE);
            }
            dup2(archivo_entrada, STDIN_FILENO);
            close(archivo_entrada);
        }
        if (comando->salida != NULL) {
```

```

        int archivo_salida = open(comando->salida, O_WRONLY | O_CREAT | O_TRUNC,
0644);

        if (archivo_salida < 0) {
            perror("Error al abrir el archivo de salida...");
            exit(EXIT_FAILURE);
        }
        dup2(archivo_salida, STDOUT_FILENO);
        close(archivo_salida);
    }
    execvp(comando->args[0], comando->args);
    perror("Error en la ejecución del programa...");
    exit(EXIT_FAILURE);
} else {
    waitpid(proceso_id, &estado, 0);
}
}

int ejecutar_pwd(struct orden* comando)
{
    char directorio[250];

    if (comando->args[1] != NULL || comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'pwd' no debe tener parámetros\n");
        return -1;
    }

    if (getcwd(directorio, sizeof(directorio)) != NULL) {
        printf("%s\n", directorio);
        return 0;
    } else {
        perror("No se pudo obtener el directorio actual");
        return -1;
    }
}

int ejecutar_cd(struct orden* comando)
{
    const char* ruta_home = getenv("HOME");

    if (comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'cd' no permite redirección.\n");
        return -1;
    }

    if (comando->args[1] == NULL) {
        if (ruta_home == NULL) {
            fprintf(stderr, "Error en la variable HOME\n");
            return -1;
        }
        if (chdir(ruta_home) != 0) {
            perror("Error al cambiar de directorio");
            return -1;
        }
    } else if (comando->args[2] != NULL) {

```



```

        fprintf(stderr, "Error: demasiados argumentos\n");
        return -1;
    } else {
        if (chdir(comando->args[1]) != 0) {
            perror("Error al cambiar de directorio");
            return -1;
        }
        return 0;
    }
}

int ejecutar_exit(struct orden* comando)
{
    if (comando->args[1] != NULL || comando->entrada != NULL || comando->salida != NULL) {
        fprintf(stderr, "Error: 'exit' no debe tener parámetros\n");
        return -1;
    }

    return 0;
}

int procesar_comando(struct orden* comando)
{
    if (strcmp("pwd", comando->args[0]) == 0) {
        return ejecutar_pwd(comando);
    } else if (strcmp("cd", comando->args[0]) == 0) {
        return ejecutar_cd(comando);
    } else if (strcmp("exit", comando->args[0]) == 0) {
        return ejecutar_exit(comando);
    } else {
        fprintf(stderr, "Comando no reconocido: %s. Se ejecutará como orden externa...\n",
comando->args[0]);
        ejecutar_orden_externa(comando);
        return 0;
    }
}

void mostrar_comando(struct orden* comando)
{
    int indice;

    for (indice = 0; comando->args[indice] != NULL; indice++) {
        printf("%s ", comando->args[indice]);
    }
    if (comando->entrada != NULL) {
        printf("< %s ", comando->entrada);
    }
    if (comando->salida != NULL) {
        printf("> %s ", comando->salida);
    }
    printf("\n");
}

int main(const int num_args, const char* args[])

```

```
{
    struct orden* comando_actual;
    int estado, continuar = 1;

    do {
        comando_actual = leer_orden("br0285> ");
        if (comando_actual->args[0] != NULL) {
            estado = procesar_comando(comando_actual);
            if (estado == 0 && strcmp("exit", comando_actual->args[0]) == 0) {
                continuar = 0;
            }
        }
    } while (continuar);

    return 0;
}
```